

Mastering React Hooks: useRef and useContext

Introduction to React Hooks

Revolutionizing Functional Components

React Hooks, introduced in version 16.8, fundamentally changed how developers write React applications. They allow functional components to manage state and side effects, eliminating the need for complex class components.

Cleaner Code: Promote more reusable and readable code structures.

Simplified Lifecycle: Manage component logic without lifecycle methods.

Focus: Today we dive into **useRef** and **useContext**.

Understanding the useRef Hook

Persisting Values Across Renders

The **useRef** hook is a powerful tool for persisting values between renders without causing the component to re-render when the value changes.

```
const ref = useRef(initialValue);
```

It returns a mutable ref object whose **.current** property is initialized to the passed argument. This object persists for the full lifetime of the component.

Key Characteristics of useRef

Mutability Without Re-renders

No Re-renders

Updating the `.current` property does not notify React to update the UI. This makes it ideal for performance-sensitive tracking of values like timer IDs or scroll positions.

Direct DOM Access

Acts as an "escape hatch" to interact directly with the Document Object Model. Essential for managing focus, text selection, or integrating with imperative third-party libraries.

Common Use Cases for useRef

Practical Applications in React

DOM Access

Interacting directly with DOM nodes for tasks like **managing focus**, text selection, or triggering animations that React doesn't handle natively.

Third-Party Libraries

Integrating with imperative APIs from external libraries (e.g., D3.js, Google Maps) that require a **direct reference** to a DOM element.

Mutable Storage

Storing values that need to persist across renders but **should not trigger re-renders**, such as timer IDs or previous state values.

Measuring Elements

Retrieving physical properties of elements, such as **width, height, or scroll position**, which are only available after the component has mounted.

Example: Accessing DOM Elements

Managing Focus Imperatively

This example demonstrates how **useRef** creates a reference to a specific DOM node.

By assigning the ref to the **ref** attribute of an input, we can access the underlying element via **inputEl.current** and call native methods like **focus()**.

```
import React, { useRef } from 'react';

function TextInputWithFocusButton() {
  const inputEl = useRef(null);

  const onClick = () => {
    // Access the DOM node directly
    inputEl.current.focus();
  };

  return (
    <>
      <input ref={inputEl} type="text" />
      <button onClick={onClick}>
        Focus the input
      </button>
    </>
  );
}
```

Example: Storing Mutable Values

Tracking Timers Without Re-renders

In this example, `useRef` is used to store an interval ID. Unlike state, updating the ref's `.current` property does not trigger a re-render.

This is essential for cleaning up side effects (like `setInterval`) without affecting the component's visual state.

```
function Timer() {
  const [count, setCount] = useState(0);
  const timerId = useRef(null);

  const start = () => {
    timerId.current = setInterval(() => {
      setCount(c => c + 1);
    }, 1000);
  };

  const stop = () => {
    clearInterval(timerId.current);
  };

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={start}>Start</button>
      <button onClick={stop}>Stop</button>
    </div>
  );
}
```

Understanding the useContext Hook

Simplifying Global State Management

The **useContext** hook provides a way to share data globally across a React application without "prop drilling"—passing props manually through every level.

- 01.** Create a context object using **createContext()**.
- 02.** Wrap components with a **Provider** to supply the value.
- 03.** Consume the value in any nested component using **useContext**.

```
// 1. Create Context
const MyContext = createContext(default);

// 2. Provide Value
<MyContext.Provider value={data}>

// 3. Consume Value
const value = useContext(MyContext);
```


Key Characteristics of useContext

Efficient Data Sharing & Reactivity

Prop Drilling Solution

Makes data accessible to deeply nested components **effortlessly**. Components can subscribe to context values without intermediate components needing to pass props.

Re-render Behavior

Any component calling **useContext** will automatically re-render whenever the context value changes. This ensures the UI is always in sync with the global state.

Common Use Cases for useContext

When to Use Context in React

Theming

Managing a user's preference for **light or dark mode** across the entire application UI without passing props through every component.

Localization

Handling **multi-language support** by providing the current locale and translation strings to deeply nested text components.

Authentication

Storing **user login status** and profile information, allowing any component to check permissions or display user-specific data.

Global Config

Sharing **application settings**, API endpoints, or feature flags that need to be accessible by various services and components.

Example: Theming with useContext

Implementing a Dark Mode Switcher

This example shows how **useContext** allows a nested component to access global state without prop drilling.

The **App** provides the theme, and **ThemedButton** consumes it directly, bypassing **Toolbar**.

```
const ThemeContext = createContext('light');

function App() {
  const [theme, setTheme] = useState('light');
  return (
    <ThemeContext.Provider value={theme}>
      <Toolbar />
      <button onClick={() => setTheme(theme === 'light' ? 'dark' : 'light')}>Toggle
    </ThemeContext.Provider>
  );
}

function Toolbar() { return <div><ThemedButton /></div>; }

function ThemedButton() {
  const theme = useContext(ThemeContext);
  return <button style={{ background: theme === 'dark' ? '#333' : '#EEE' }}>I am sty
}
```

Comparing useRef and useState

Choosing the Right Hook for Your Data

Feature	useRef	useState
Trigger Re-render	No	Yes
Mutability	Mutable (<code>.current</code>)	Immutable (via setter)
Access Method	Synchronous	Asynchronous (scheduled)
Primary Use Case	DOM access, mutable values	UI state, data rendering

Best Practices and Considerations

Optimizing Hook Usage

✔ useRef Guidelines

Avoid using **useRef** for data that needs to be rendered in the UI; use **useState** instead.

Don't over-rely on DOM manipulation; keep your code **declarative** whenever possible.

⚠ useContext Performance

Avoid using context for **high-frequency** state changes (e.g., mouse tracking) to prevent excessive re-renders.

For complex global state, consider dedicated libraries like **Redux** or **Zustand**.

Conclusion

Mastering React's Essential Hooks

useRef

A safe haven for **mutable values** that don't trigger re-renders and a direct bridge to the **DOM** for imperative actions.

useContext

An elegant solution to **prop drilling**, enabling efficient sharing of global data like themes and auth status across the component tree.

Build cleaner, more performant, and maintainable React applications.